Scientific Computing Exercise Set 1

Michelangelo Grasso (15840417) & Ilias-Panagiotis Sofianos (15889378) & Francesco Tiepolo (13319817)

***Repository**: https://github.com/francescotiepolo/scientific_computing_assignment1.git*

I. Introduction

NUMERICAL methods are crucial for solving complex physical problems that cannot always be approached analytically. In this report, we aim to investigate two cases: the vibration of a string that is described by the one-dimensional wave equation, and the time-dependent diffusion equation that illustrates how the concentration varies over time in a medium.

The first section of the report delves into the discretization of the wave equation through the use of a central difference scheme to simulate the motion of a vibrating string under different initial conditions. The second section explores the diffusion equation of a two-dimensional domain, comparing numerical and analytical results, and analyzing the efficiency of iterative methods such as Jacobi, Gauss-Seidel and Successive Over Relaxation (SOR).

II. Theory

A. Vibrating String

We approach the topic of finite differences starting with the analysis of a one-dimensional wave equation, whose domain is - in this case - a string:

$$\frac{\partial^2 \Psi}{\partial t^2} = c^2 \frac{\partial^2 \Psi}{\partial x^2} \tag{1}$$

where $\Psi(x,t)$ is the vibration amplitude expressed as a function of position and time. We assume fixed boundaries, such that $\Psi(x=0,t) = \Psi(x=L,t) = 0$, where L is the length of the string. We will here discretize Equation 1 by finite differences with a central difference scheme.

We proceed by dividing the spatial domain in N intervals, so that $\Delta x = \frac{L}{N}$. The function Ψ is now Taylor expanded (forward) around the point $x + \Delta x$, obtaining:

$$\Psi_{i+1} = \Psi_i + \Delta x \frac{\partial \Psi}{\partial x} + \frac{\Delta x^2}{2!} \frac{\partial^2 \Psi}{\partial x^2} + O(\Delta x^3) \tag{2}$$

where Ψ_i=$\Psi(x_i,t)$ and $x_i = i\Delta x$. We do the same in the backward direction:

$$\Psi_{i-1} = \Psi_i - \Delta x \frac{\partial \Psi}{\partial x} + \frac{\Delta x^2}{2!} \frac{\partial^2 \Psi}{\partial x^2} + O(\Delta x^3) \tag{3}$$

Summing Equations 2 and 3 leads to the following[1]:

$$\Psi_{i+1} + \Psi_{i-1} = 2\Psi_i + \Delta x^2 \frac{\partial^2 \Psi}{\partial x^2} + O(\Delta x^4)$$

which can be rearranged to formally derive the central difference approximation for $\partial^2\Psi/\partial x^2$ (second order accurate):

$$\frac{\partial^2 \Psi}{\partial x^2} \approx \frac{\Psi_{i+1} - 2\Psi_i + \Psi_{i-1}}{\Delta x^2} \tag{4}$$

The same discretization process is also applied at the time level, to obtain:

$$\frac{\partial^2 \Psi}{\partial t^2} \approx \frac{\Psi_i^{n+1} - 2\Psi_i^n + \Psi_i^{n-1}}{\Delta t^2} \tag{5}$$

where the time domain is divided into intervals of size Δt, $\Psi_i^n = \Psi(x_i, t_n)$ and $t_n = n\Delta t$. Substituting Equations 4 and 5 into Equation 1 and rearranging its terms allows us to derive the full discretization for the wave equation[2]:

$$\Psi_i^{n+1} = 2\Psi_i^n - \Psi_i^{n-1} + c^2 \frac{\Delta t^2}{\Delta x^2} (\Psi_{i+1}^n - 2\Psi_i^n + \Psi_{i-1}^n) \tag{6}$$

To finalize the problem at hand, the initial conditions must be specified. Considering that the string is at rest at $t = 0$, the analysis is here focused on three different initial conditions:

i $\Psi(x, t=0) = \sin(2\pi x)$
ii $\Psi(x, t=0) = \sin(5\pi x)$
iii $\Psi(x, t=0) = \sin(5\pi x)$ if $1/5 < x < 2/5$, else $\Psi = 0$

B. Time Independent Diffusion Equation

To describe diffusion, Adolf Fick proposed a relation which stated that the flux of particles due to a concentration gradient obeys:

$$J = -D \frac{dC}{dx} \tag{7}$$

[1]Here, the last term is $O(\Delta x^4)$ and not $O(\Delta x^3)$ because the Δx^3 terms (and any other term with x raised to the power of any odd-exponent) cancel out when the forward and backward approximations are summed together

[2]For a more extensive mathematical analysis of the complete discretization process, refer to the designed PDF file in the repository mentioned above

[1] Where J is the flux (mol/m^{2}s), D is the diffusion coefficient (m^2/s) and C is the concentration (mol/m^3). The negative sign suggests that diffusion happens in the decreasing concentration direction. This is also known as Fick's First Law and is applicable to steady-state conditions, where there is no change in the concentration profile over time. However, it is common that diffusion is a non steady state process, meaning that concentration is altered over time. To incorporate this fact into the mathematical framework, Fick introduced his second law that is illustrated by the following equation that governs the time evolution of concentration in a system:

$$\frac{\partial C}{\partial t} = D\frac{\partial^2 C}{\partial x^2} \tag{8}$$

In the 2D special domain, the Laplacian operator is used which turns the equation to

$$\frac{\partial C}{\partial t} = D\nabla^2 C \quad \Rightarrow \quad \frac{\partial C}{\partial t} = D\frac{\partial^2 C}{\partial x^2} + D\frac{\partial^2 C}{\partial y^2} \tag{9}$$

Since it is often impractical to solve the equations analytically, numerical methods such as the finite difference method are utilized. This method discretizes both space and time, and approximates derivatives using difference equations. The computational domain is turned to a grid with grid spacing:

$$\delta x = \delta y = \frac{1}{N}$$

for a grid size $N \times N$ and a time step δt that must satisfy the stability condition [2]:

$$\frac{4\delta t D}{\delta x^2} \leq 1$$

An appropriate approach is the explicit finite difference scheme, which uses values from the previous time step to update the concentration at each grid point. To derive the scheme we first approximate the second derivative of the concentration in terms of the space coordinates:

$$\nabla^2 C = \frac{\partial^2 C}{\partial x^2} + \frac{\partial^2 C}{\partial y^2}$$

$$\frac{\partial^2 C}{\partial x^2} \approx \frac{C(x+\delta x, y, t) - 2C(x, y, t) + C(x-\delta x, y, t)}{\delta x^2}$$

$$\frac{\partial^2 C}{\partial y^2} \approx \frac{C(x, y+\delta y, t) - 2C(x, y, t) + C(x, y-\delta y, t)}{\delta y^2}$$

By combining these formulas, we get:

$$\nabla^2 C \approx \frac{C^k_{i+1,j} + C^k_{i-1,j} + C^k_{i,j+1} + C^k_{i,j-1} - 4C^k_{i,j}}{\delta x^2}$$

Now we use forward Euler method for time integration:

$$\begin{aligned} \frac{C^{k+1}_{i,j} - C^k_{i,j}}{\delta t} &= D\nabla^2 C^k_{i,j} \\ \Rightarrow \quad C^{k+1}_{i,j} &= C^k_{i,j} + \frac{\delta t D}{\delta x^2}\left(C^k_{i+1,j} + C^k_{i-1,j}\right) \\ &\quad + \frac{\delta t D}{\delta x^2}\left(C^k_{i,j+1} + C^k_{i,j-1} - 4C^k_{i,j}\right) \end{aligned} \tag{10}$$

Focusing on the steady state, the time-independent diffusion equation $\nabla^2 c = 0$ is solved directly. Assuming the boundary conditions, spatial discretization and the 5-points stencil for the second order derivatives remain unchanged,

$$c^{k+1}_{i,j} = c^k_{i,j} + \frac{\delta t D}{\delta x^2}(c^k_{i+1,j} + c^k_{i-1,j} + c^k_{i,j+1} + c^k_{i,j-1} - 4c^k_{i,j}) \tag{11}$$

transforms to the following set of finite difference equations

$$\frac{1}{4}(c_{i+1,j} + c_{i-1,j} + c_{i,j+1} + c_{i,j-1}) = c_{i,j} \tag{12}$$

for all values of (i, j). Below are notable iterative methods used to solve the equations. The advantage over direct methods lies in their efficient use of memory.

1) Jacobi Iteration: The Jacobi Iteration method is the solution of the time-dependent equation (11) with the maximum allowed time step (($\frac{\delta t D}{\delta x^2} = \frac{1}{4}$)), leading to:

$$c^{k+1}_{i,j} = \frac{1}{4}(c^k_{i+1,j} + c^k_{i-1,j} + c^k_{i,j+1} + c^k_{i,j-1}) \tag{13}$$

where k represents the k-th iteration.

Compared to the time dependent case, where a time step and the total time of simulation is defined, in an iterative method a stopping condition is needed. The stopping condition indicates convergence for all values of (i, j), and is defined as follows.

$$\delta \equiv max_{ij}|c^{k+1}_{i,j} - c^k_{i,j}| < \epsilon \tag{14}$$

with ϵ being small.

2) Gauss-Seidel Iteration: A small improvement over the Jacobi iteration is made in the Gauss-Seidel method by using the new value calculated at each iteration as soon as it is calculated. This leads to:

$$c^{k+1}_{i,j} = \frac{1}{4}(c^k_{i+1,j} + c^{k+1}_{i-1,j} + c^k_{i,j+1} + c^{k+1}_{i,j-1}) \tag{15}$$

3) Successive Over Relaxation: The Successive Over Relaxation is obtained from Gauss-Seidel through an over-correction of the new iterate, leading to:

$$c^{k+1}_{i,j} = \frac{\omega}{4}(c^k_{i+1,j} + c^{k+1}_{i-1,j} + c^k_{i,j+1} + c^{k+1}_{i,j-1}) + (1-\omega)c^k_{i,j} \tag{16}$$

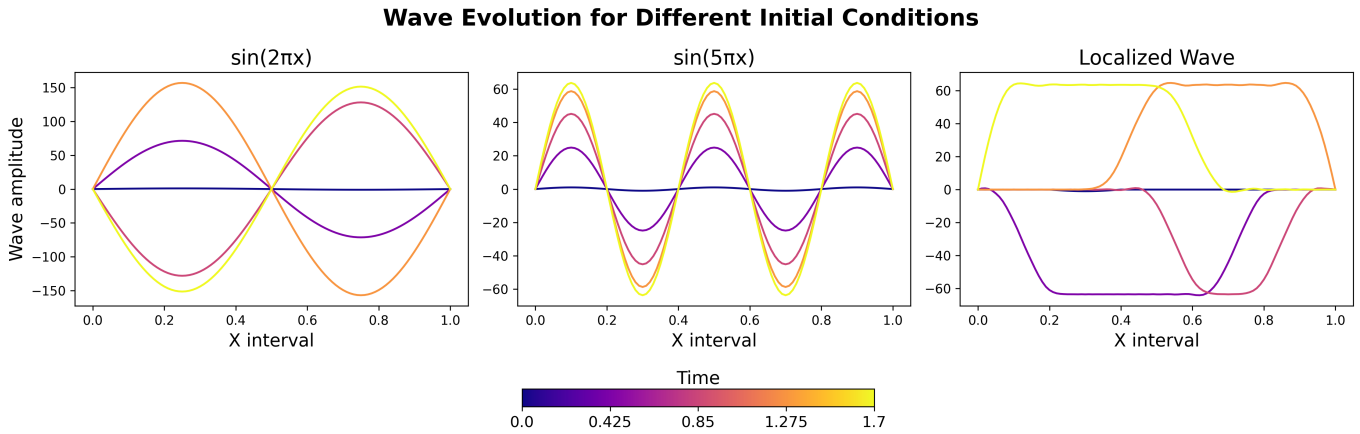

Fig. 1: The image shows the string at different time points for the three different initial conditions. $L = 1$, $N = 100$, $c = 1$ and $\Delta t = 0.001$. The lines referring to $t = 0$ (the darkest ones) might look the same, but this is simply due to the initial conditions' amplitude values (Ψ) only spanning in the domain $[0, 1]$, making it hard to notice the difference between the three cases when plotted together with the following time steps, whose amplitude ranges in much larger domains

The method converges only for $0 < \omega < 2$. For $\omega < 1$ the method is called under-relaxation. The new value is then the weighted average of the Gauss-Seidel method and the previous value. For $\omega = 1$, the method is equal to the Gauss-Seidel iteration. For the diffusion problem of the report, the optimal ω minimizing the number of iterations needed for convergence lies in the range $1.7 < \omega < 2$. The exact value depends on the grid size N.

III. Methods

Regarding the string simulation, having specified the initial and boundary conditions, Equation 6 can be fed to a computer program which, by iterating through every time step, computes the amplitude value at every spatial step. We run the latter simulation in Python (using Numpy) for which the report will later present the results. In other words, the program will compute Ψ_i^{n+1} based on the amplitude of the - spatially - nearest two points (for the boundaries, this is given by the previously set conditions) and the previous value on the same spatial coordinate (Ψ_i^{n-1}); thus, the first computer iteration will be possible because the string will be generated at $t = 0$ according to one of the above initial conditions.

The two-dimensional time-dependent diffusion equation, $\frac{\partial c}{\partial t} = D\nabla^2 c$ was approached by using an explicit finite difference scheme. Firstly, the spatial domain was discretized into a 50×50 grid with periodic boundary conditions in the x-axis direction and fixed boundary conditions in the y-axis direction ($c(x, y = 1; t) = 1$ and $c(x, y = 0; t) = 0$). The initial conditions set the concentration to the value of one ($c = 1$) at the top boundary and zero everywhere else.

The finite difference update formula is:

$$c_{i,j}^{k+1} = c_{i,j}^k + \frac{\delta t D}{\delta x^2}(c_{i+1,j}^k + c_{i-1,j}^k + c_{i,j+1}^k + c_{i,j-1}^k - 4c_{i,j}^k) \tag{17}$$

which has to follow the stability condition:

$$\frac{4\delta t D}{\delta x^2} \leq 1. \tag{18}$$

The time step δt is set as $\delta t = 0.25\delta x^2/D$ so that it fulfills the stability condition, since $D = 1$.

For the boundary conditions of the domain, we will use the discretized equation 10

$$c_{i,j}^{k+1} = c_{i,j}^k + \frac{\delta t D}{\delta x^2}(c_{i+1,j}^k + c_{i-1,j}^k + c_{i,j+1}^k + c_{i,j-1}^k - 4c_{i,j}^k) \tag{19}$$

We establish the following rules:

- Top and bottom boundaries ($y = 1$ and $y = 0$): $c(x, y = 1, t) = 1$, $c(x, y = 0, t) = 0$ (fixed points that are not updated)
- Periodic boundary in x-direction: $c(x = 0, y, t) = c(x = 1, y, t)$

In finite difference terms this is expressed as:

$$c_{i,0}^k = 0, \quad c_{i,N}^k = 1, \quad \forall i, \quad 0 \leq i \leq N \tag{20}$$

$$c_{0,j}^k = c_{N,j}^k, \quad c_{N+1,j}^k = c_{1,j}^k, \quad \forall j, \quad 0 \leq j \leq N \tag{21}$$

Adjusting for periodicity:

$$c_{0,j}^{k+1} = c_{0,j}^k + \frac{\delta t D}{\delta x^2}(c_{1,j}^k + c_{N-1,j}^k + c_{0,j+1}^k + c_{0,j-1}^k - 4c_{0,j}^k) \tag{22}$$

$$c_{N,j}^{k+1} = c_{N,j}^k + \frac{\delta t D}{\delta x^2}(c_{1,j}^k + c_{N-1,j}^k + c_{N,j+1}^k + c_{N,j-1}^k - 4c_{N,j}^k) \tag{23}$$

The analytical solution for the concentration levels as a function of y and t is calculated using the error function [3]:

$$c(y,t) = \sum_{i=0}^{\infty} \left[\text{erfc}\left(\frac{1-y+2i}{2\sqrt{Dt}}\right) - \text{erfc}\left(\frac{1+y+2i}{2\sqrt{Dt}}\right) \right] \tag{24}$$

The simulation was implemented in Python. Numpy was used for array manipulation, Matplotlib for plotting, and Numba for optimizing performance. Snapshots of the concentration field were captured at selected time points and compared with the analytical solution. Finally, an animated plot was generated to visualize the time evolution of the diffusion process.

IV. Results and Discussion

A. String Behavior

Figure 1 plots the results from Equation 6 at several time points. Firstly, we can notice how the boundary conditions are fixed, observing that the start and end of the string are still at 0 for every time step. Secondly, and most importantly, it should be seen how different initial conditions (plotted by the darkest line) result in significantly diverse behaviors of the string throughout the simulation time.

The plotting of these results[3] was made possible by the discretization method of finite differences by central difference approximation, as explained in the previous sections; this allowed to convert the wave PDE to a mathematical expression which could be handled by a computer, iterating through every time step and, in each one of these, computing Ψ for every x based on their (temporally) previous and (spatially) neighbors values, as shown in Equation 6.

[3] Also presented in an animated version in the linked repository

B. Discretization of Diffusion Equation

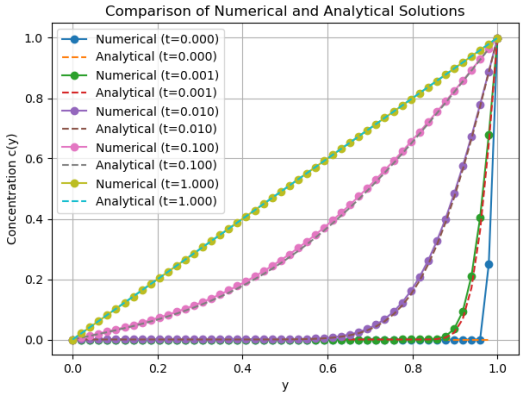

Fig. 3: Comparison of Numerical and Analytical Solutions for different points in time. With dotted lines the analytical solutions is plotted, while with continuous lines the numerical.

In the first graph, we plotted the concentration versus the y coordinate of the space domain for different points in time. Specifically, this plot describes how the concentration level changes on the y-axis for a specific time frame. What we observe from the results is that the dotted lines that represent the analytical solution are in a perfect accordance with the numerical solution. Therefore, it is reasonable to assume that our simulation is correct, since it reproduced the expected theoretical results.

Regarding the second plot, we observe for the same time points the diffusion process in the concentration field. Initially, the concentration is zero everywhere except for the top boundary, which is set to $c = 1$. As time progresses, the high concentration region at the top diffuses downward, thus creating a smooth gradient from top (yellow) to bottom (blue). By $t = 1$, the system approaches a near linear gradient, consistent with the expected equilibrium solution where concentration transitions steadily from 1 at the top to 0 at the bottom.

In terms of time-independent iterative methods, the Jacobi, Gauss-Seidel and SOR methods are used for a grid of size N = 50. To test the methods, they are compared to the analytical solution to ensure linear dependence to the concentration of y [4].

In order to compare the efficiency of the three methods, a plot showing the number of iterations k needed for convergence as the tolerance value is changed is shown below in figure 4. In particular, for SOR three different values of ω are used to have a visual representation of its effects on the number of iterations to convergence.

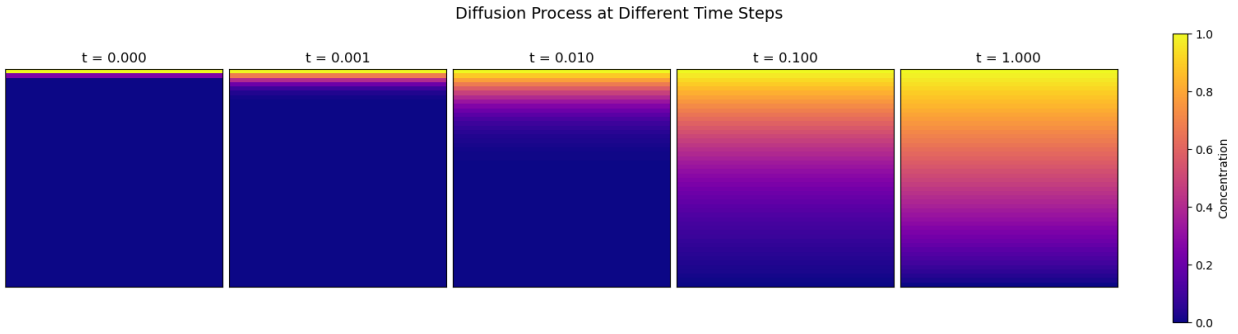

Fig. 2: Time evolution of the concentration at different time points: $t = 0$, $t = 0.001$, $t = 0.01$, $t = 0.1$, and $t = 1$.

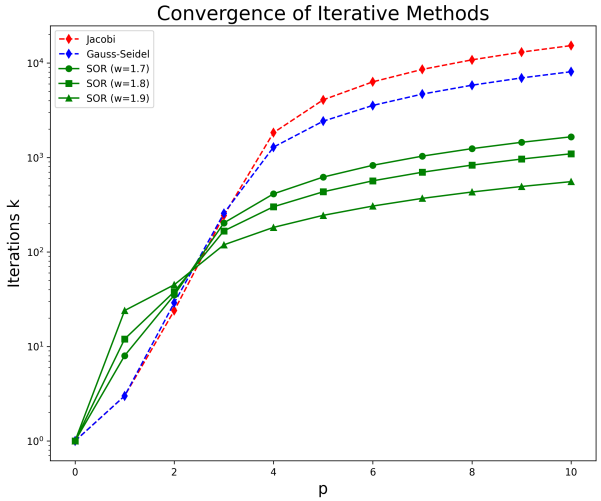

Fig. 4: Number of iterations k to convergence plotted against different values of tolerance, where p refers to 10^{-p}. As p increases, more iterations are necessary for convergence. for lower p values Jacobi and Gauss-Seidel converge faster than SOR, while for larger values of p SOR outperforms the other alternatives, especially so with $\omega = 1.9$

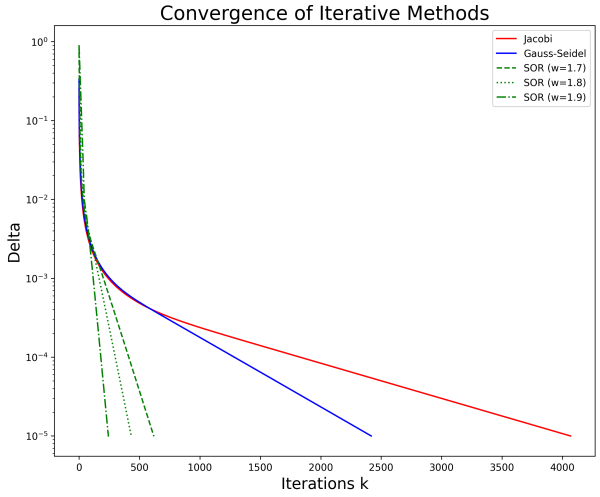

Fig. 5: Plot of number of iterations k needed for convergence through the SOR method for grids of different sizes.The optimal ω are represented by colored dots. The plot indicates that as grid size increases, the optimal ω increases towards 2 and the number of iterations k needed to reach convergence also increases.

As mentioned in the background knowledge, the performance of SOR depends on the value of ω, and for the diffusion problem at hand, the optimal ω lies between 1.7 and 2. Since this value might also depend on grid size, figure 5 shows a plot of the number of iterations needed for convergence through the SOR method for grids of different sizes. Furthermore, the optimal ω are represented by colored dots, of which the values are specified in the legend.

An interesting aspect to explore is the possibility of including objects in the form of sinks in the initial grid. If a point (i, j) is part of an object, the concentration is just 0. When a cell comes into contact with the object, its value is set to zero [5]. The effects are investigated with a square and two triangles, which are presented below in figure 6.

V. Conclusion

By approaching the solutions of the wave and diffusion equations numerically using finite difference methods, we illustrate key aspects of their behavior. For the vibrating string problem, the central difference scheme managed to capture wave propagation with varying initial conditions. Specifically, the sinusoidal initial conditions, produced smooth waveforms, while the piecewise one led to localized oscillations. These results validate

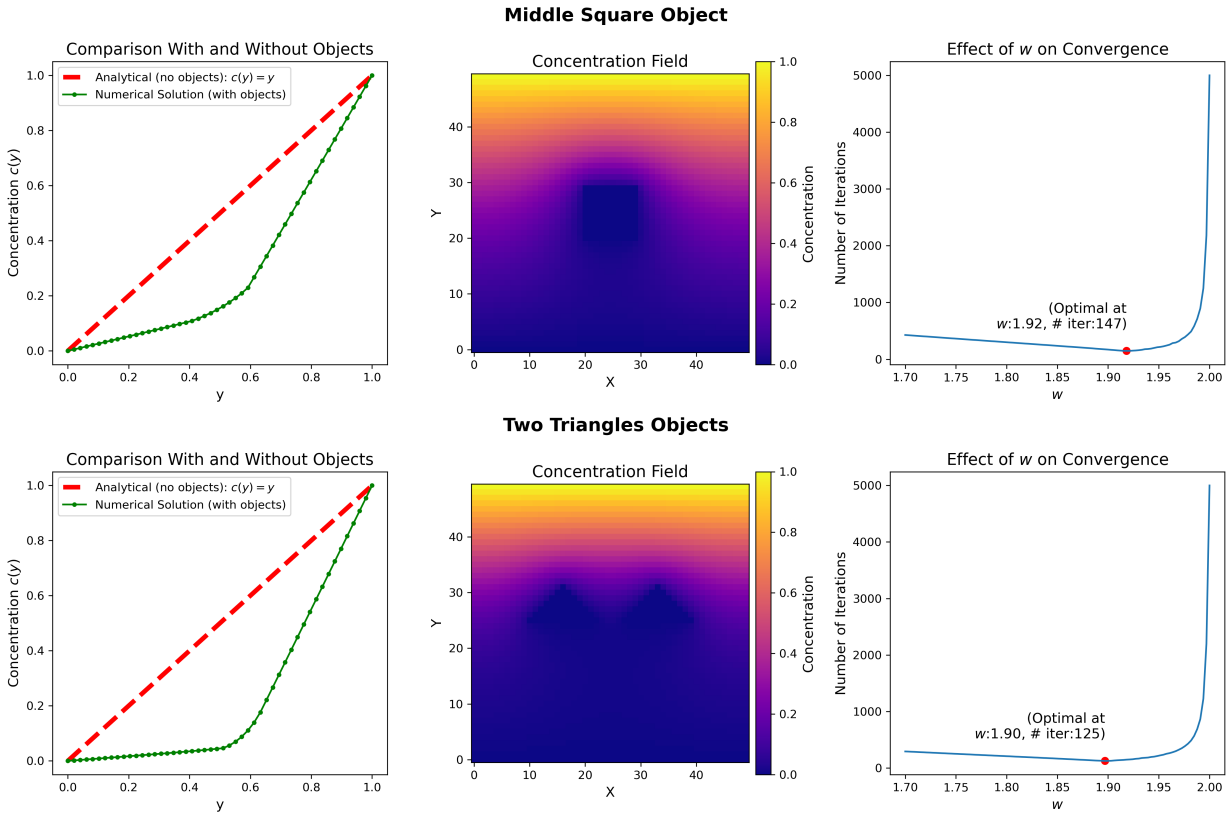

Fig. 6: The first plot compares the numerical solution (green) with the object to the analytical solution (red) without any object to visualize the changes in the concentration profile vs y. The second plot shows the two-dimensional concentration distribution, with the object placed in the middle. It is noticeable how the object disrupts the gradient. In the third plot the number of iterations needed for convergence for different ω is represented, where a red dot indicates the ω value at which the lowest number of iterations is needed for convergence.

the method's property of handling different initial conditions and boundary constraints.

Regarding the diffusion equation, the explicit finite difference method matched the analytical solution of the concentration profile. The system reached a steady-state solution and confirmed the accuracy of the method. By comparing the data derived from the iterative solvers (Jacobi, Gauss-Seidel and SOR), we showed that SOR was the most efficient at tighter tolerance levels, with an optimal relaxation parameter around $\omega = 1.7 - 2$. This emphasizes the trade-off between accuracy and computational cost.

REFERENCES

[1] Peter Atkins and Julio de Paula. *Physical Chemistry for the Life Sciences*. Oxford University Press, 2006.

[2] Shin-Taek Jeong. "Stability of Finite Difference Schemes on the Diffusion Equation with Discontinuous Coefficients". In: 2018. URL: https://api.semanticscholar.org/CorpusID:32296632.

[3] Hubert Chanson. "Diffusion: Basic Theory". In: *Environmental Hydraulics of Open Channel Flows*. Ed. by Hubert Chanson. Burlington, MA: Elsevier Butterworth-Heinemann, 2004. ISBN: 9780750661652. DOI: 10.1016/B978-075066165-2.50030-8.

[4] K. R. James and W. Riha. "Convergence Criteria for Successive Overrelaxation". In: *SIAM Journal on Numerical Analysis* 12.2 (1975), pp. 137–143.

[5] Peter S. Huyakorn et al. "A three-dimensional finite-element model for simulating water flow in variably saturated porous media". In: *Water Resources Research* 22.13 (1986), pp. 1790–1808.